vertex shader are compiled and created by passing the shader type as a parameter to theglCreateShader() method.

A check is performed to confirm whether the required shader binary (GL_NUM_SHADER_BINARY_FORMATS) and shader source (GL_SHADER_COMPILER) is supported.

```
bool GlesCube::InitGL()
{
    GLint linked = GL_FALSE;

    // Calculate the color of each fragment passed to the fragment shader
    GLuint fragShader = glCreateShader(GL_FRAGMENT_SHADER);

    // Calculate the vertex properties and values that are passed to the vertex shader
    GLuint vertShader = glCreateShader(GL_VERTEX_SHADER);

    GLint numFormats = 0;
    glGetIntegerv(GL_NUM_SHADER_BINARY_FORMATS, &numFormats);

    GLboolean shaderCompiler = GL_FALSE;
    glGetBooleanv(GL_SHADER_COMPILER, &shaderCompiler);
```

b. The next step is to load the shader objects. There are 2 ways to achieve this:

The first approach is to use the `glShaderBinary()` method to load the shader objects - vertex and fragment - by loading their respective precompiled shader binaries.

```
    if(0 < numFormats)
    {
        GLboolean compiled = GL_FALSE;
        GLint* formats = new GLint[numFormats];
        glGetIntegerv(GL_SHADER_BINARY_FORMATS, formats);
        for(GLint i = 0; i < numFormats; i++)
        {
            glShaderBinary(1, &fragShader, formats[i] , frag-
```